

ROS机器人理论与实践

第2章 ROS通信机制

孔庆群 智能科学与技术系
qingqun.kong@ia.ac.cn

教材：ROS机器人理论与实践（第1版） 张新钰 清华大学出版社



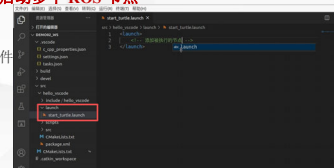
1.4.3 launch文件演示

2

需求：一个程序中可能需要启动多个节点，比如：ROS 内置的小乌龟案例，如果要控制乌龟运动，要启动多个窗口，分别启动 `roscore`、乌龟界面节点、键盘控制节点。如果每次都调用 `roslaunch` 逐一启动，显然效率低下，如何优化？

解决方法：使用 launch 文件，一次性启动多个 ROS 节点

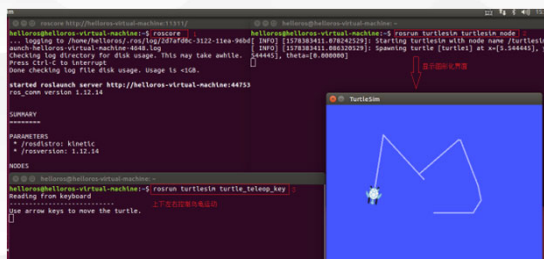
1. 选定功能包右击 --> 添加 launch 文件夹
2. 选定 launch 文件夹右击 --> 添加 launch 文件
3. 编辑 launch 文件内容
4. 运行 launch 文件
5. 运行结果：一次性启动了多个节点



`roslaunch hello_vscode start_turtle.launch`

1.4.3 launch文件演示

3



1.4.3 launch文件演示

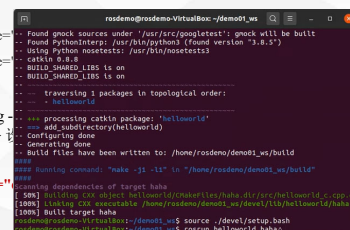
4

编辑 launch 文件内容

```
<launch>
  <node pkg="turtlesim" type="turtlesim1" />
  <node pkg="turtlesim" type="turtlesim2" />
</launch>
```

node --> 包含的某个节点; pkg --> 为节点命名; output --> 输出

```
<launch>
  <node pkg="helloworld" type="helloworld" />
</launch>
```



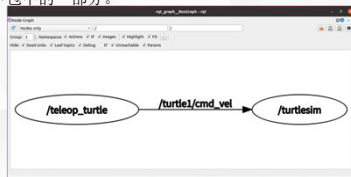
注意：只要有launch标签就会自动启动roscore

1.5.3 ROS计算图

5

前面介绍的是ROS文件结构，是磁盘上ROS程序的存储结构，是静态的，而ros程序运行之后，不同的节点之间是错综复杂的，ROS中提供了一个实用的工具：**rqt_graph**。

rqt_graph能够创建一个显示当前系统运行情况的动态图形。ROS分布式系统中不同进程需要进行数据交互，计算图可以以点对点的网络形式表现数据交互过程。rqt_graph是rqt程序包中的一部分。



1.5.3 ROS计算图

6

计算图安装

如果前期把所有的功能包（package）都已经安装完成，则直接在终端窗口中输入

```
roslaunch rqt_graph rqt_graph
```

如果未安装则在终端（terminal）中输入

```
$ sudo apt install ros-<distro>-rqt
```

```
$ sudo apt install ros-<distro>-rqt-common-plugins
```

用自己的ROS版本名称（比如:kinetic、melodic、Noetic等）来替换掉<distro>。

例如当前版本是 Noetic,就在终端窗口中输入

```
$ sudo apt install ros-noetic-rqt
```

```
$ sudo apt install ros-noetic-rqt-common-plugins
```

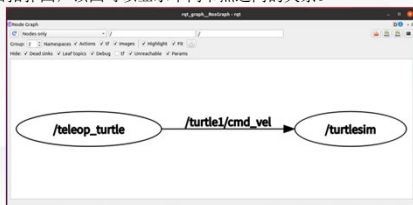
1.5.3 ROS计算图

7

计算图演示

首先，按照前面所示，运行案例(ROS内置的小乌龟案例)

然后，启动新终端，键入:rqt_graph 或 roslaunch rqt_graph rqt_graph，可以看到类似下图的网络拓扑图，该图可以显示不同节点之间的关系。



1.2 ROS 安装

8

如果采用虚拟机安装 ubuntu，再安装 ROS 的话，大致流程如下：

- ① 安装虚拟机软件(比如: virtualbox 或 VMware); ---视频007
- ② 使用虚拟机软件虚拟一台主机; ---视频008
- ③ 在虚拟主机上安装 ubuntu 20.04; ---视频009-013
- ④ 在 ubuntu 上安装 ROS;
- ⑤ 测试 ROS 环境是否可以正常运行。

>>> 1.2 ROS 安装 9

Ubuntu 安装完毕后, 就可以安装 ROS 操作系统了, 大致步骤如下:---视频014

- ① 配置ubuntu的软件和更新;
- ② 设置安装源;
- ③ 设置key;
- ④ 安装;
- ⑤ 配置环境变量。

>>> 1.2.4 安装ROS 10

1.配置ubuntu的软件和更新

配置ubuntu的软件和更新, 允许安装不经认证的软件。

首先打开“软件和更新”对话框, 具体可以在 Ubuntu 搜索按钮中搜索。

打开后按照下图进行配置 (确保勾选了"restricted", "universe, " 和 "multiverse.")



>>> 1.2.4 安装ROS 11

2. 设置安装源

官方默认安装源:

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'
```

或来自国内清华的安装源

```
sudo sh -c 'echo "deb http://mirrors.tuna.tsinghua.edu.cn/ros/ubuntu/ $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'
```

或来自国内中科大的安装源

```
sudo sh -c 'echo "deb http://mirrors.ustc.edu.cn/ros/ubuntu/ $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'
```

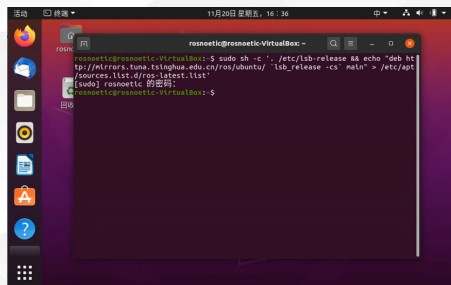
PS:

回车后, 可能需要输入管理员密码

建议使用国内资源, 安装速度更快。

>>> 1.2.4 安装ROS 12

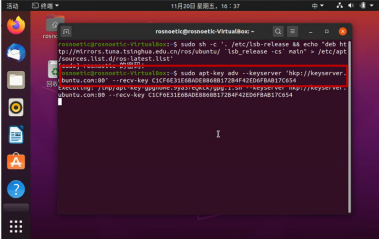
2. 设置安装源



1.2.4 安装ROS 13

3. 设置key

sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654

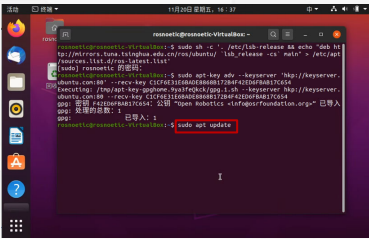


1.2.4 安装ROS 14

4. 安装

首先需要更新 apt(以前是 apt-get, 官方建议使用 apt 而非 apt-get). apt 是用于从互联网仓库搜索、安装、升级、卸载软件或操作系统的工具。

sudo apt update

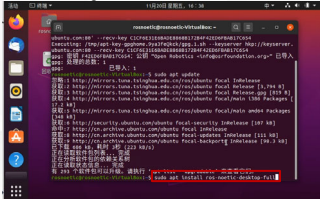


1.2.4 安装ROS 15

4. 安装

然后, 再安装所需类型的 ROS. ROS 多个类型-Desktop-Full、Desktop、ROS-Base。这里介绍较为常用的Desktop-Full(官方推荐)安装. ROS, rqt, rviz, robot-generic libraries, 2D/3D simulators, navigation and 2D/3D perception

sudo apt install ros-noetic-desktop-full



1.2.4 安装ROS 16

5. 配置环境变量

echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
source ~/.bashrc

卸载

如果需要卸载ROS可以调用如下命令:

sudo apt remove ros-noetic-*

注意: 在 ROS 版本 noetic 中无需构建软件包的依赖关系, 没有rosdep的相关安装与配置。

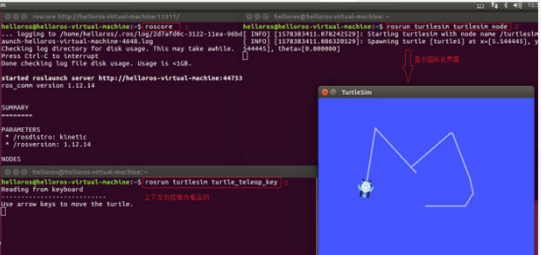
另请参考: <http://wiki.ros.org/noetic/Installation/Ubuntu>。

1.2.5 测试ROS

17

输入命令后结果如下

ROS 内置了一些小程序，可以通过运行这些小程序以检测 ROS 环境是否可以正常运行



1.3.1 HelloWorld实现简介

18

编写 ROS 程序，在控制台输出文本: Hello World，分别使用 C++ 和 Python 实现 ROS 中涉及的编程语言以C++和Python为主，ROS中的大多数程序两者都可以实现 ROS中的程序即便使用不同的编程语言，实现流程也大致类似，以当前HelloWorld 程序为例，实现流程大致如下：

- ① 先创建一个工作空间；
- ② 再创建一个功能包；
- ③ 编辑源文件；
- ④ 编辑配置文件；
- ⑤ 编译并执行。

上述流程中，C++和Python只是在步骤3和步骤4的实现细节上存在差异，其他流程基本一致。

1.3.1 HelloWorld实现简介

19

1.创建工作空间并初始化

```
mkdir -p 自定义空间名称/src
cd 自定义空间名称
catkin_make
```

视频17—4 '40

2.进入 src 创建 ros 包并添加依赖

```
cd src
catkin create_pkg 自定义ROS包名
roscpp rospy std_msgs
```

功能包

创建一个工作空间以及一个 src 子目录，然后再进入工作空间调用 catkin_make 命令编译。

功能包依赖于 roscpp、rospy 与 std_msgs，其中 roscpp 是使用 C++ 实现的库，而 rospy 则是使用 python 实现的库，std_msgs 是标准消息库，创建 ROS 功能包时，一般都会依赖这三个库实现。



1.3.2 HelloWorld(C++ 版)

20

3.编辑源文件

视频18---1

```
#include "ros/ros.h" //包含头文件
int main(int argc, char *argv[])
{
    //执行 ros 节点初始化
    ros::init(argc, argv, "hello");
    //创建 ros 节点句柄(非必须)
    ros::NodeHandle n;
    //控制台输出 hello world
    ROS_INFO("hello world");
    return 0;
}
```

4.编辑 ros 包下的 CmakeList.txt 文件

```
add_executable(步骤3的源文件名
src/步骤3的源文件名.cpp
)
target_link_libraries(步骤3的源文件名
${catkin_LIBRARIES}
)
```

>>> 1.3.2 HelloWorld(C++版)

21

3.进入工作空间目录并编译

```
cd 自定义空间名称
catkin_make
```

4.执行

```
先启动命令行1:
roscore

再启动命令行2:
cd 工作空间
source ./devel/setup.bash //将当前工作空间刷新到当前窗口下面的环境变量，然后就可以执行功能包了
roslaunch 包名 C++节点

命令行输出: HelloWorld!
```

问题：如果是在任意窗口下执行怎么办呢？ --视频19 2'

>>> 1.3.3 HelloWorld(Python版)

22

1.编辑 python 文件

```
#!/usr/bin/env python

"""
    Python 版 HelloWorld
"""
import rospy //导包

if __name__ == "__main__":
    rospy.init_node("Hello") //初始化ros节点
    rospy.loginfo("Hello World!!!!") //输出日志
```

2.为 python 文件添加可执行权限

```
chmod +x 自定义文件名.py
```

3.编辑 ros 包下的 CamkeList.txt 文件

```
catkin_install_python(PROGRAMS scripts/自定义文件名.py
                        DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
                        )
```

>>> 1.3.2 HelloWorld(C++版)

23

4.进入工作空间目录并编译

```
cd 自定义空间名称
catkin_make
```

5.执行

```
先启动命令行1:
roscore

再启动命令行2:
cd 工作空间
source ./devel/setup.bash
roslaunch 包名 python节点

命令行输出: HelloWorld!
```

1.作业名称：VSCode+ROS 集成开发环境实战

作业要求：

a) 配置 VSCode 的 ROS 开发环境，安装必备插件并解决中文乱码、代码提示等问题；

b) 在 VSCode 中完成上述“Helloworld”的编写、编译、调试；

c) Helloworld部分需要提交一份从头操作的录频视频；第三组需要课堂现场进行演示

2.作业名称：编写 launch 文件，实现小乌龟程序的一键启动;利用计算图输出小乌龟程序网络拓扑图

作业要求：

需要提交一份从头操作的录频视频；第三组需要课堂现场进行演示

3.作业名称：基于c++和python实现发布方以固定频率发送一段文本（自定义数据），订阅方订阅消息并将消息内容打印输出

作业要求：

需要提交一份从头操作的录频视频；第四组需要课堂现场进行演示

目录/Contents	
01	话题通信
02	服务通信
03	参数服务器
04	常用命令
05	通信机制实操
06	通信机制比较
07	本章小结

>>> 2.0 绪论

26

机器人是一种高度复杂的系统性实现，在机器人上可能集成各种传感器(雷达、摄像头、GPS等)以及运动控制实现。为了解耦合，在ROS中每一个功能点都是一个单独的进程，每一个进程都是独立运行的。更确切的讲，ROS是进程（也称为Nodes）的分布式框架，因为这些进程甚至还可分布于不同主机，不同主机协同工作，从而分散计算压力。不过随之也有一个问题: **不同的进程是如何通信的？也即不同进程间如何实现数据交换的？**

>>> 2.0 绪论

27

ROS 中的基本通信机制主要有如下三种实现策略:

- 话题通信(发布订阅模式)
- 服务通信(请求响应模式)
- 参数服务器(参数共享模式)

>>> 2.0 绪论

28

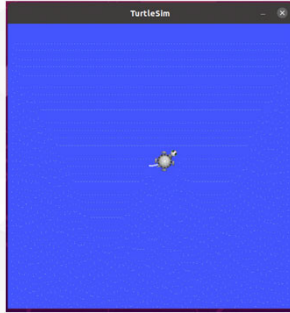
本章的主要内容就是介绍各个通信机制的应用场景、理论模型、代码实现以及相关操作命令。本章预期达成学习目标如下:

- 能够熟练介绍ROS中常用的通信机制
- 能够理解ROS中每种通信机制的理论模型
- 能够以代码的方式实现各种通信机制对应的案例
- 能够熟练使用ROS中的一些操作命令
- 能够独立完成相关实操案例

2.0 绪论

29

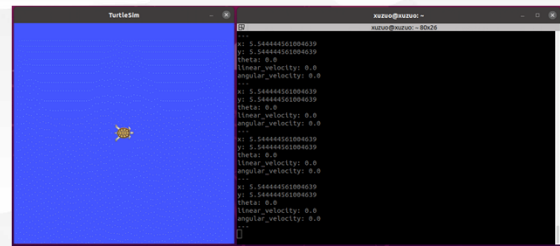
1.话题演示案例: 控制小乌龟做圆周运动



2.0 绪论

30

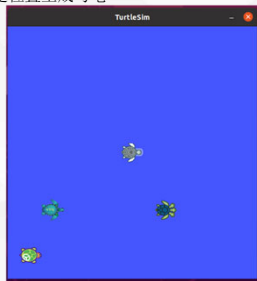
1.话题演示案例:获取乌龟位姿



2.0 绪论

31

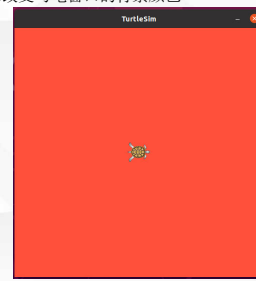
2.服务演示案例:在指定位置生成乌龟



2.0 绪论

32

3. 参数演示案例: 改变乌龟窗口的背景颜色





>>> 2.1 话题通信基本概念

34

话题通信是ROS中使用频率最高的一种通信模式，话题通信是基于**发布订阅**模式的，也即:一个节点发布消息，另一个节点订阅该消息。

场景：机器人在执行导航功能，使用的传感器是激光雷达，机器人会采集激光雷达感知到的信息并计算，然后生成运动控制信息驱动机器人底盘运动。

- 以激光雷达信息的采集处理为例，在 ROS 中有一个节点需要时时的发布当前雷达采集到的数据，导航模块中也有节点会订阅并解析雷达数据。
- 以运动消息的发布为例，导航模块会根据传感器采集的数据时时的计算出运动控制信息并发布给底盘，底盘也可以有一个节点订阅运动信息并最终转换成控制电机的脉冲信号。

像雷达、摄像头、GPS.... 等等一些传感器数据的采集，也都是使用了话题通信

话题通信适用于不断更新的数据传输相关的应用场景

>>> 2.1 话题通信基本概念

35

概念

以发布订阅的方式实现不同节点之间数据交互的通信模式。

作用

用于不断更新的、少逻辑处理的数据传输场景。

案例

实现最基本的发布订阅模型，发布方以固定频率发送一段文本，订阅方接收文本并输出

>>> 2.1 话题通信基本概念

36

案例

1. 实现最基本的发布订阅模型，发布方以固定频率发送一段文本，订阅方接收文本并输出

```
roscore http://rosdemo-virtualbox:11311/6b24  rosdemo@rosdemo-virtualbox:~/demo02_ws$ cat
ros_comm version 1.15.8
INFO [1608289346.55657239]: 发布的消息:hello -- 57
INFO [1608289346.55646547]: 发布的消息:hello -- 58
INFO [1608289346.55710447]: 发布的消息:hello -- 59
INFO [1608289346.55690892]: 发布的消息:hello -- 60
SUMMARY
INFO [1608289346.55688711]: 发布的消息:hello -- 61
INFO [1608289346.55657848]: 发布的消息:hello -- 62
PARAMETERS
 * /roscpp_rate: roscpp
 * /rosversion: 1.15.8
INFO [1608289346.55678493]: 发布的消息:hello -- 63
INFO [1608289346.55704678]: 发布的消息:hello -- 64
INFO [1608289346.55682202]: 发布的消息:hello -- 65
INFO [1608289346.55717244]: 发布的消息:hello -- 66
INFO [1608289346.55681249]: 我听见:hello -- 57
INFO [1608289346.55701312]: 我听见:hello -- 58
INFO [1608289346.55709086]: 我听见:hello -- 59
INFO [1608289346.55726232]: 我听见:hello -- 60
INFO [1608289346.55717244]: 我听见:hello -- 61
INFO [1608289346.55690990]: 我听见:hello -- 62
INFO [1608289346.55691607]: 我听见:hello -- 63
INFO [1608289347.55621182]: 我听见:hello -- 64
INFO [1608289348.55681565]: 我听见:hello -- 65
INFO [1608289349.55733795]: 我听见:hello -- 66
started core service [/roscpp]
```

2.1 话题通信基本概念

37

案例

2.实现对自定义消息的发布与订阅

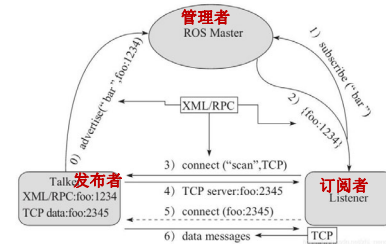
```

rosdemo@rosdemo-VirtualBox: ~/demo02_ws
$ cat /usr/share/rosdemo-VirtualBox/11111/Adm34
INFO [1606289607.073999981] 姓名:张三, 年龄:27, 身高:1.73
INFO [1606289608.074000096] 姓名:张三, 年龄:27, 身高:1.73
INFO [1606289609.074000199] 姓名:张三, 年龄:27, 身高:1.73
INFO [1606289610.073999932] 姓名:张三, 年龄:27, 身高:1.73
INFO [1606289611.074000218] 姓名:张三, 年龄:27, 身高:1.73
INFO [1606289612.073999912] 姓名:张三, 年龄:27, 身高:1.73
INFO [1606289613.073999917] 姓名:张三, 年龄:27, 身高:1.73
INFO [1606289614.073999917] 姓名:张三, 年龄:27, 身高:1.73
INFO [1606289615.074000024] 姓名:张三, 年龄:27, 身高:1.73
INFO [1606289616.074000088] 姓名:张三, 年龄:27, 身高:1.73
$ rosrun roscpp demo02_ws_45451
INFO [1606289607.073999981] 接收到的人的信息是:张三,27,1.73
INFO [1606289608.074000096] 接收到的人的信息是:张三,27,1.73
INFO [1606289609.073999932] 接收到的人的信息是:张三,27,1.73
INFO [1606289610.073999932] 接收到的人的信息是:张三,27,1.73
INFO [1606289611.074000218] 接收到的人的信息是:张三,27,1.73
INFO [1606289612.073999912] 接收到的人的信息是:张三,27,1.73
INFO [1606289613.073999917] 接收到的人的信息是:张三,27,1.73
INFO [1606289614.073999917] 接收到的人的信息是:张三,27,1.73
INFO [1606289615.074000024] 接收到的人的信息是:张三,27,1.73
INFO [1606289616.074000088] 接收到的人的信息是:张三,27,1.73

```

2.1.1 理论模型

38



ROS Master 负责保管 Talker 和 Listener 注册的信息，并匹配话题相同的 Talker 与 Listener，帮助 Talker 与 Listener 建立连接，连接建立后，Talker 可以发布消息，且发布的信息会被 Listener 订阅。

2.1.1 理论模型

39

0.Talker注册

Talker启动后，会通过RPC在 ROS Master 中注册自身信息，其中包含所发布消息的话题名称。ROS Master 会将节点的注册信息加入到注册表中。

1.Listener注册

Listener启动后，也会通过RPC在 ROS Master 中注册自身信息，包含需要订阅消息的话题名称。ROS Master 会将节点的注册信息加入到注册表中。

2.ROS Master实现信息匹配

ROS Master 会根据注册表中的信息匹配Talker 和 Listener，并通过 RPC 向 Listener 发送 Talker 的 RPC 地址信息。

3.Listener向Talker发送请求

Listener 根据接收到的 RPC 地址，通过 RPC 向 Talker 发送连接请求，传输订阅的话题名称、消息类型以及通信协议(TCP/UDP)。

2.1.1 理论模型

40

4.Talker确认请求

Talker 接收到 Listener 的请求后，也是通过 RPC 向 Listener 确认连接信息，并发送自身的 TCP 地址信息。

5.Listener与Talker建立连接

Listener 根据步骤4 返回的消息使用 TCP 与 Talker 建立网络连接。

6.Talker向Listener发送消息

连接建立后，Talker 开始向 Listener 发布消息。

>>> 2.1.2 话题通信基本操作(C++)

45

需求：编写发布订阅实现，要求发布方以10HZ(每秒10次)的频率发布文本消息，订阅方订阅消息并将消息内容打印输出。-视频40

1.发布方

实现的关键点：
1.发送方
2.接收方
3.数据(此处为普通文本)

PS: 二者需要设置相同的话题 消息发布方: 循环发布信息HelloWorld 后缀数字编号

实现流程:

- 1.包含头文件
- 2.初始化 ROS 节点命名(唯一)
- 3.创建ROS 句柄
- 4.创建 发布者 对象
- 5.编写发布逻辑并发布数据 */

```

// 发布方
#include <ros/ros.h>
#include <std_msgs/String.h>
using namespace std;

int main(int argc, char *argv[])
{
    // 1. 初始化 ROS 节点
    ros::init(argc, argv, "publisher");
    // 2. 创建 ROS 句柄
    ros::NodeHandle nh;
    // 3. 创建发布者对象
    ros::Publisher pub = nh.advertise<String>("topic", 10);
    // 4. 发布消息
    std_msgs::String msg;
    while (ros::ok())
    {
        msg.data = "HelloWorld";
        pub.publish(msg);
        ros::spinOnce();
    }
    return 0;
}

```

>>> 2.1.2 话题通信基本操作(C++)

46

需求：编写发布订阅实现，要求发布方以10HZ(每秒10次)的频率发布文本消息，订阅方订阅消息并将消息内容打印输出。-视频40

1.发布方

实现的关键点：
1.发送方
2.接收方
3.数据(此处为普通文本)

PS: 二者需要设置相同的话题 消息发布方: 循环发布信息HelloWorld 后缀数字编号

实现流程:

- 1.包含头文件
- 2.初始化 ROS 节点命名(唯一)
- 3.创建ROS 句柄
- 4.创建 发布者 对象
- 5.编写发布逻辑并发布数据 */

```

// 发布方
#include <ros/ros.h>
#include <std_msgs/String.h>
using namespace std;

int main(int argc, char *argv[])
{
    // 1. 初始化 ROS 节点
    ros::init(argc, argv, "publisher");
    // 2. 创建 ROS 句柄
    ros::NodeHandle nh;
    // 3. 创建发布者对象
    ros::Publisher pub = nh.advertise<String>("topic", 10);
    // 4. 发布消息
    std_msgs::String msg;
    while (ros::ok())
    {
        msg.data = "HelloWorld";
        pub.publish(msg);
        ros::spinOnce();
    }
    return 0;
}

```

>>> 2.1.2 话题通信基本操作(C++)

47

需求：编写发布订阅实现，要求发布方以10HZ(每秒10次)的频率发布文本消息，订阅方订阅消息并将消息内容打印输出。-视频40

1.发布方

实现的关键点：
1.发送方
2.接收方
3.数据(此处为普通文本)

PS: 二者需要设置相同的话题 消息发布方: 循环发布信息HelloWorld 后缀数字编号

实现流程:

- 1.包含头文件
- 2.初始化 ROS 节点命名(唯一)
- 3.创建ROS 句柄
- 4.创建 发布者 对象
- 5.编写发布逻辑并发布数据 */

```

// 发布方
#include <ros/ros.h>
#include <std_msgs/String.h>
using namespace std;

int main(int argc, char *argv[])
{
    // 1. 初始化 ROS 节点
    ros::init(argc, argv, "publisher");
    // 2. 创建 ROS 句柄
    ros::NodeHandle nh;
    // 3. 创建发布者对象
    ros::Publisher pub = nh.advertise<String>("topic", 10);
    // 4. 发布消息
    std_msgs::String msg;
    while (ros::ok())
    {
        msg.data = "HelloWorld";
        pub.publish(msg);
        ros::spinOnce();
    }
    return 0;
}

```

编写数据逻辑

>>> 2.1.2 话题通信基本操作(C++)

48

需求：编写发布订阅实现，要求发布方以10HZ(每秒10次)的频率发布文本消息，订阅方订阅消息并将消息内容打印输出。

2.订阅方

实现的关键点：
1.发送方
2.接收方
3.数据(此处为普通文本)

消息订阅方: 订阅话题并打印接收到的消息

实现流程:

- 1.包含头文件
- 2.初始化 ROS 节点命名(唯一)
- 3.实例化 ROS 句柄
- 4.实例化 订阅者 对象
- 5.处理订阅的消息回调函数
- 6.设置循环调用回调函数 */

```

// 订阅方
#include <ros/ros.h>
#include <std_msgs/String.h>
using namespace std;

// 回调函数
void callback(const std_msgs::String::ConstPtr& msg)
{
    cout << "Received: " << msg->data << endl;
}

int main(int argc, char *argv[])
{
    // 1. 初始化 ROS 节点
    ros::init(argc, argv, "subscriber");
    // 2. 创建 ROS 句柄
    ros::NodeHandle nh;
    // 3. 创建订阅者对象
    ros::Subscriber sub = nh.subscribe<String>("topic", 10, callback);
    // 4. 启动回调函数
    ros::spin();
    return 0;
}

```

>>> 2.1.2 话题通信基本操作(C++)

49

需求：编写发布订阅实现，要求发布方以10HZ(每秒10次)的频率发布文本消息，订阅方订阅消息并将消息内容打印输出。

2.订阅方

实现的关键点:

- 1.发送方
- 2.接收方
- 3.数据(此处为普通文本)

消息订阅方: 订阅话题并打印接收到的消息

实现流程:

- 1.包含头文件
- 2.初始化 ROS 节点命名(唯一)
- 3.实例化 ROS 句柄
- 4.实例化订阅者对象
- 5.处理订阅的消息(回调函数)
- 6.设置循环调用回调函数

```

1 // 2.初始化 ROS 节点:
2 ros::init(argc, argv, "subscriber");
3 // 3.创建句柄对象:
4 ros::NodeHandle nh;
5 // 4.创建订阅者对象:
6 ros::Subscriber sub = nh.subscribe("topic", 10, callback);
7 // 5.处理订阅的消息:
8 void callback(const std_msgs::String& msg) {
9     ROS_INFO("收到订阅的消息: %s", msg.data.c_str());
10 }
11
12 int main(int argc, char* argv[])
13 {
14     // 2.初始化 ROS 节点:
15     ros::init(argc, argv, "subscriber");
16     // 3.创建句柄对象:
17     ros::NodeHandle nh;
18     // 4.创建订阅者对象:
19     ros::Subscriber sub = nh.subscribe("topic", 10, callback);
20     // 5.处理订阅的消息:
21     void callback(const std_msgs::String& msg) {
22         ROS_INFO("收到订阅的消息: %s", msg.data.c_str());
23     }
24     return 0;
25 }

```

>>> 2.1.2 话题通信基本操作(C++)

50

需求：编写发布订阅实现，要求发布方以10HZ(每秒10次)的频率发布文本消息，订阅方订阅消息并将消息内容打印输出。

2.订阅方

实现的关键点:

- 1.发送方
- 2.接收方
- 3.数据(此处为普通文本)

消息订阅方: 订阅话题并打印接收到的消息

实现流程:

- 1.包含头文件
- 2.初始化 ROS 节点命名(唯一)
- 3.实例化 ROS 句柄
- 4.实例化订阅者对象
- 5.处理订阅的消息(回调函数)
- 6.设置循环调用回调函数

```

1 int main(int argc, char* argv[])
2 {
3     // 2.初始化 ROS 节点:
4     ros::init(argc, argv, "subscriber");
5     // 3.创建句柄对象:
6     ros::NodeHandle nh;
7     // 4.创建订阅者对象:
8     ros::Subscriber sub = nh.subscribe("topic", 10, callback);
9     // 5.处理订阅的消息:
10    void callback(const std_msgs::String& msg) {
11        ROS_INFO("收到订阅的消息: %s", msg.data.c_str());
12    }
13    return 0;
14 }

```

>>> 2.1.2 话题通信基本操作(C++)

51

3. 配置CMakeLists.txt

```

1 add_executable(Hello_pub
2   src/Hello_pub.cpp
3 )
4 add_executable(Hello_sub
5   src/Hello_sub.cpp
6 )
7
8 target_link_libraries(Hello_pub
9   ${catkin_LIBRARIES}
10 )
11 target_link_libraries(Hello_sub
12   ${catkin_LIBRARIES}
13 )

```

>>> 2.1.2 话题通信基本操作(C++)

52

4.执行

- 1.启动 roscore;
- 2.启动发布节点;
- 3.启动订阅节点。

5.注意

补充0:

vscode 中的 main 函数声明 int main(int argc, char const *argv[]){}，默认生成 argv 被 const 修饰，需要去除该修饰符

补充1:

ros/ros.h No such file or directory
检查 CMakeList.txt find_package 出现重复,删除内容少的即可
参考资料:<https://answers.ros.org/question/237494/fatal-error-rosrosh-no-such-file-or-directory/>

补充2:

订阅时，第一条数据丢失
原因: 发送第一条数据时，publisher 还未在 roscore 注册完毕
解决: 注册后，加入休眠 ros::Duration(3.0).sleep(); 延迟第一条数据的发送

>>> 2.1.3 话题通信基本操作(Python)

53

需求：编写发布订阅实现，要求发布方以10HZ(每秒10次)的频率发布文本消息，订阅方订阅消息并将消息内容打印输出。

在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个：

- 1.发布方
- 2.接收方
- 3.数据(此处为普通文本)

流程：

- 1.编写发布方实现；
- 2.编写订阅方实现；
- 3.为python文件添加可执行权限；
- 4.编辑配置文件；
- 5.编译并执行。

>>> 2.1.3 话题通信基本操作(Python)

54

1.发布方

实现流程:

- 1.导包
- 2.初始化 ROS 节点名(唯一)
- 3.实例化 发布者 对象
- 4.组织被发布的数据，并编写逻辑发布数据

2.订阅方

实现流程:

- 1.导包
- 2.初始化 ROS 节点名(唯一)
- 3.实例化 订阅者 对象
- 4.处理订阅的消息(回调函数)
- 5.设置循环调用回调函数

>>> 2.1.3 话题通信基本操作(Python)

55

1.发布方

实现流程:

- 1.导包
- 2.初始化 ROS 节点名(唯一)
- 3.实例化 发布者 对象
- 4.组织被发布的数据，并编写逻辑发布数据

```
if __name__ == "__main__":  
    # 2. 初始化ROS节点：  
    rospy.init_node("sanDai") #传入节点名称  
    # 3. 创建发布者对象：  
    pub = rospy.Publisher("che",String,queue_size=10)  
    # 4. 编写发布逻辑并发布数据。  
    # 创建数据  
    msg = String()  
    #使用循环发布数据  
    while not rospy.is_shutdown():  
        msg.data = "hello"  
        #发布数据  
        pub.publish(msg)
```

>>> 2.1.3 话题通信基本操作(Python)

56

1.发布方

实现流程:

- 1.导包
- 2.初始化 ROS 节点名(唯一)
- 3.实例化 发布者 对象
- 4.组织被发布的数据，并编写逻辑发布数据

```
# 2. 初始化ROS节点：  
rospy.init_node("sanDai") #传入节点名称  
# 3. 创建发布者对象：  
pub = rospy.Publisher("che",String,queue_size=10)  
# 4. 编写发布逻辑并发布数据。  
# 创建数据  
msg = String()  
#设置发布频率  
rate = rospy.Rate(1)  
#设置计数器  
count = 0  
#使用循环发布数据  
while not rospy.is_shutdown():  
    count = count + 1  
    msg.data = "hello" + str(count)  
    #发布消息  
    pub.publish(msg)  
    rospy.loginfo("发布的消息:%s",msg.data)  
    rate.sleep()
```

>>> 2.1.3 话题通信基本操作(Python)

57

2.订阅方

实现流程:

1.导包

2.初始化 ROS 节点(命名唯一)

3.实例化 订阅者 对象

4.处理订阅的消息(回调函数)

5.设置循环调用回调函数

```
def doMsg(msg):
    rospy.loginfo("我订阅的数据:%s",msg.data)

if __name__ == "__main__":
    # 2. 初始化ROS节点;
    rospy.init_node("huaTua")
    # 3. 创建订阅者对象;
    sub = rospy.Subscriber("che",String,doMsg,queue_size=10)
    # 4. 回调函数处理数据;
    # 5.spin()
    rospy.spin()
```

>>> 2.1.3 话题通信基本操作(Python)

58

3.添加可执行权限

终端下进入 scripts 执行:chmod +x *.py

4.配置CMakeLists.txt

```
catkin_install_python(PROGRAMS
  scripts/talker_p.py
  scripts/listener_p.py
  DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)
```

5.执行

1.启动 roscore;

2.启动发布者节点;

3.启动订阅节点。

>>> 2.1.4 话题通信自定义msg

59

在 ROS 通信协议中，数据载体是一个较为重要组成部分，ROS 中通过 std_msgs 封装了一些原生的数据类型,比如:String、Int32、Int64、Char、Bool、Empty.... 但是，这些数据一般只包含一个 data 字段，结构的单一意味着功能上的局限性，当传输一些复杂的数据，比如:激光雷达的信息...std_msgs 由于描述性较差而显得力不从心，这种场景下可以使用自定义的消息类型

msgs只是简单的文本文件，每行具有字段类型和字段名称，可以使用的字段类型有: int8, int16, int32, int64 (或者无符号类型: uint*) float32, float64 string time, duration other msg files variable-length array[] and fixed-length array[C]

ROS中还有一种特殊类型: Header，标头包含时间戳和ROS中常用的坐标帧信息。会经常看到msg文件的第一行具有Header标头。

>>> 2.1.4 话题通信自定义msg

60

需求:创建自定义消息，该消息包含人的信息:姓名、身高、年龄等。-视频52

流程:

1.按照固定格式创建 msg 文件

2.编辑配置文件

3.编译生成可以被 Python 或 C++ 调用的中间文件

1.定义msg文件

功能包下新建 msg 目录，添加文件 Person.msg

2. 编辑配置文件

➢ package.xml中添加编译依赖与执行依赖

➢ CMakeLists.txt编辑 msg 相关配置

```
string name
uint16 age
float64 height
```


2.2 服务通信基本概念

65

服务通信也是ROS中一种极其常用的通信模式，服务通信是基于请求响应模式的，是一种应答机制。也即：一个节点A向另一个节点B发送请求，B接收处理请求并产生响应结果返回给A。

场景：机器人巡逻过程中，控制系统分析传感器数据发现可疑物体或人... 此时需要拍摄照片并留存。

- 一个节点需要向相机节点发送拍照请求，相机节点处理请求，并返回处理结果。

服务通信更适用于对实时性有要求、具有一定逻辑处理的应用场景。

2.2 服务通信基本概念

66

概念

以请求响应的方式实现不同节点之间数据交互的通信模式。

作用

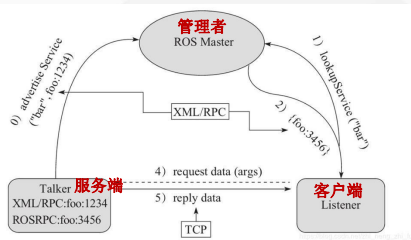
用于偶然的、对实时性有要求、有一定逻辑处理需求的数据传输场景。

案例

实现两个数字的求和，客户端节点，运行会向服务器发送两个数字，服务器端节点接收两个数字求和并将结果响应回客户端。

2.2.1 理论模型

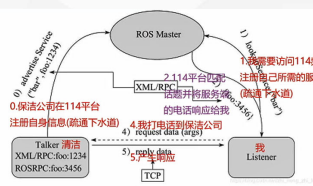
67



ROS Master 负责保管 Server 和 Client 注册的信息，并匹配话题相同的 Server 与 Client，帮助 Server 与 Client 建立连接，连接建立后，Client 发送请求信息，Server 返回响应信息。

2.2.1 理论模型

68



角色

- 1. 话题
- 2. 服务端
- 3. 客户端
- 4. 数据载体

角色 ---> 流程 ---> 注意

- 角色
- 1.master ----> 管理者 (114平台)
 - 2.Server ----> 服务端 (服务公司)
 - 3.Client ----> 客户端 (我)
- 流程
- master 会根据话题实现 Server 和 Client 的连接
- 注意:
- 1. 保证顺序，客户端发起请求时，服务端需要已经启动
 - 2. 客户端和服务端都可以存在多个。

0. Server注册

Server 启动后，会通过RPC在 ROS Master 中注册自身信息，其中包含提供的服务的名称。ROS Master 会将节点的注册信息加入到注册表中。

1. Client注册

Client 启动后，也会通过RPC在 ROS Master 中注册自身信息，包含需要请求的服务的名称。ROS Master 会将节点的注册信息加入到注册表中。

2.ROS Master实现信息匹配

ROS Master 会根据注册表中的信息匹配Server和 Client，并通过 RPC 向 Client 发送 Server 的 TCP 地址信息。

3. Client发送请求

Client 根据步骤2 响应的信息，使用 TCP 与 Server 建立网络连接，并发送请求数据。

4. Server发送响应

Server 接收、解析请求的数据，并产生响应结果返回给 Client。

注意1:客户端请求被处理时，需要保证服务器已经启动

注意2:服务端和客户端都可以存在多个

需求: 服务通信中，客户端提交两个整数至服务端，服务端求和并响应结果到客户端，请创建服务器与客户端通信的数据载体。

srv 文件内的可用数据类型与 msg 文件一致，且定义 srv 实现流程与自定义 msg 实现流程类似：

流程:

1. 按照固定格式创建srv文件
2. 编辑配置文件
3. 编译生成中间文件

1.定义srv文件

服务通信中，数据分成两部分，请求与响应，在 **srv 文件中请求和响应使用---分割**，实现如下：

功能包下新建 srv 目录，添加 xxx.srv 文件，内容：

```
# 客户端请求时发送的两个数字
int32 num1
int32 num2
---
# 服务器响应发送的数据
int32 sum
```

2.编辑配置文件

- package.xml中添加编译依赖与执行依赖
- CMakeLists.txt编辑 srv 相关配置

2.2.2 服务通信自定义 srv

73

package.xml中添加编译依赖与执行依赖

```
<build_depend>message_generation</build_depend>
```

```
<exec_depend>message_runtime</exec_depend>
```

CMakeLists.txt编辑 srv 相关配置

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
# 生成消息时依赖于 std_msgs
generate_messages(
  DEPENDENCIES
  std_msgs
)
# 执行时依赖
catkin_package(
  # INCLUDE_DIRS include
  # LIBRARIES demo02_talker_listener
  CATKIN_DEPENDS roscpp rospy
  std_msgs message_runtime
  # DEPENDS system_lib
)
## 配置 srv 源文件
add_service_files(
  FILES
  AddInts.srv
)
```

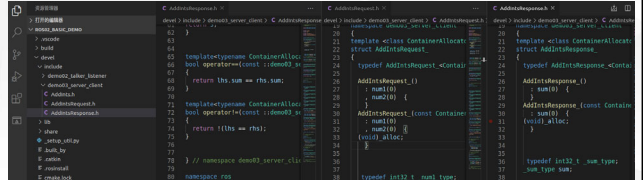
2.2.2 服务通信自定义 srv

74

3.编译

编译后的中间文件查看:

C++ 需要调用的中间文件(.../工作空间/devel/include/包名/xxx.h)



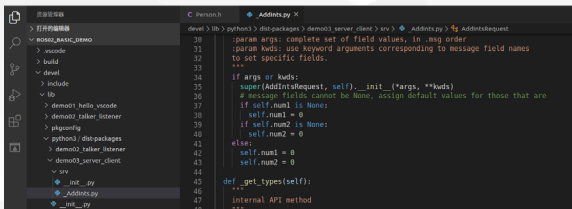
2.2.2 服务通信自定义 srv

75

3.编译

编译后的中间文件查看:

Python 需要调用的中间文件(.../工作空间/devel/lib/python3/dist-packages/包名/srv)



后续调用相关 srv 时,是从这些中间文件调用的

2.2.3 服务通信自定义srv调用(C++)

76

需求: 编写服务通信, 客户端提交两个整数至服务端, 服务端求和并响应结果到客户端。

在模型实现中, ROS master 不需要实现, 而连接的建立也已经被封装了, 需要关注的关键点有三个:

1. 服务端
2. 客户端
3. 数据

流程:

1. 编写服务端实现;
2. 编写客户端实现;
3. 编辑配置文件;
4. 编译并执行。。

>>> 2.2.3 服务通信自定义srv调用 (C++)

77

需求：编写服务通信，客户端提交两个整数至服务端，服务端求和并响应结果到客户端。

1.服务端

```
/*
服务端实现:
1.包含头文件
2.初始化 ROS 节点
3.创建 ROS 句柄
4.创建 服务 对象
5.回调函数处理请求并产生响应
6.由于 请求 有多个，需要调用
ros::spin()
*/
```

2.客户端

```
/*
客户端实现:
1.包含头文件
2.初始化 ROS 节点
3.创建 ROS 句柄
4.创建 客户端 对象
5.请求服务，接收响应
*/
```

>>> 2.2.3 服务通信自定义srv调用 (C++和Python)

78

3. 配置CMakeLists.txt

```
add_executable(AddInts_Server src/AddInts_Server.cpp)
add_executable(AddInts_Client src/AddInts_Client.cpp)

add_dependencies(AddInts_Server ${PROJECT_NAME}_gencpp)
add_dependencies(AddInts_Client ${PROJECT_NAME}_gencpp)

target_link_libraries(AddInts_Server
  ${catkin_LIBRARIES}
)
target_link_libraries(AddInts_Client
  ${catkin_LIBRARIES}
)
```

>>> 2.2.3 服务通信自定义srv调用 (C++和Python)

79

4.执行

流程：

1. 启动服务:roslaunch 包名 服务
2. 调用客户端:roslaunch 包名 客户端 参数1 参数2

结果：

会根据提交的数据响应相加后的结果。

5.注意

如果先启动客户端，那么会导致运行失败

优化：

在客户端发送请求前添加:client.waitForExistence();

或:ros::service::waitForService("AddInts");

这是一个阻塞式函数，只有服务启动成功后才会继续执行

>>> 2.2.3 服务通信自定义srv调用 (Python)

80

需求：编写服务通信，客户端提交两个整数至服务端，服务端求和并响应结果到客户端。

在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个：

- 1.服务端
- 2.客户端
- 3.数据

流程：

- 1.编写服务端实现；
- 2.编写客户端实现；
- 3.为python文件添加可执行权限；
- 4.编辑配置文件；
- 5.编译并执行。

>>> 2.2.3 服务通信自定义srv调用 (Python)

81

1.服务端

实现流程:

- 1.导包
- 2.初始化 ROS 节点命名(唯一)
- 3.创建服务对象
- 4.回调函数处理请求并产生响应
- 5.spin 函数

2.客户端

实现流程:

- 1.导包
- 2.初始化 ROS 节点命名(唯一)
- 3.创建请求对象
- 4.发送请求
- 5.接收并处理响应

>>> 2.2.3 服务通信自定义srv调用 (Python)

82

3.添加可执行权限

终端下进入 scripts 执行:chmod +x *.py

4.配置CMakeLists.txt

```
catkin_install_python(PROGRAMS
  scripts/AddInts_Server_p.py
  scripts/AddInts_Client_p.py
  DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)
```

5.执行

- 需要先启动服务:roslaunch 包名 服务
- 然后再调用客户端 :roslaunch 包名 客户端 参数1 参数2。

>>> 2.2.3 服务通信自定义srv调用 (C++和Python)效果

83

```
rosdemo@rosdemo-VirtualBox: ~/demo02_ws
$ rosrun http://rosdemo-VirtualBox:11311/
//rosdemo-VirtualBox:36871/
ros_comm version 1.15.8

SUMMARY
=====
PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.15.8

NODES
auto-starting new master
process[master]: started with
pid [17248]
ROS_MASTER_URI=http://rosdemo-
VirtualBox:11311/

setting /run_id to ch761ef2-32
bb-11eb-b515-4f5292e6a65f
process[rosout-1]: started wit
pid [17254]
started core service [/rosout]
```

03

参数服务器

>>> 2.3 参数服务器基本概念

85

参数服务器在ROS中主要用于实现不同节点之间的数据共享。参数服务器相当于是一个独立于所有节点的一个公共容器，可以将数据存储在容器中，被不同的节点调用，当然不同的节点也可以往其中存储数据。

场景：导航实现时，会进行路径规划，比如：全局路径规划，设计一个从出发点到达目标点的大致路径。本地路径规划，会根据当前路况生成时的行进路径。

- 路径规划时，需要参考小车的尺寸，我们可以将这些尺寸信息存储到参数服务器，全局路径规划节点与本地路径规划节点都可以从参数服务器中调用这些参数。

参数服务器，一般适用于存在数据共享的一些应用场景。

>>> 2.3 参数服务器基本概念

86

概念

以共享的方式实现不同节点之间数据交互的通信模式。

作用

存储一些多节点共享的数据，类似于全局变量。

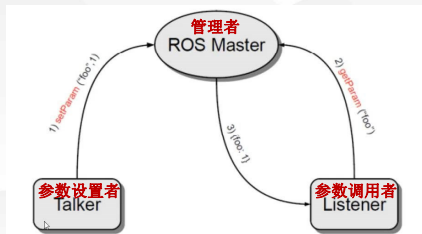
案例

实现参数增删改查操作。

>>> 2.3.1 理论模型

87

参数服务器实现是最为简单的，ROS Master 作为一个公共容器保存参数，Talker 可以向容器中设置参数，Listener 可以获取参数。



>>> 2.3.1 理论模型

88

1.Talker 设置参数

Talker 通过 RPC 向参数服务器发送参数(包括参数名与参数值)，ROS Master 将参数保存到参数列表中。

2.Listener 获取参数

Listener 通过 RPC 向参数服务器发送参数查找请求，请求中包含要查找的参数名。

3.ROS Master 向 Listener 发送参数值

ROS Master 根据步骤2请求提供的参数名查找参数值，并将查询结果通过 RPC 发送给 Listener。

注意:参数服务器不是为高性能而设计的，因此最好用于存储静态的非二进制的简单数据

>>> 2.3.2 参数操作

89

需求：实现参数服务器参数的增删改查操作。

在 C++ 中实现参数服务器数据的增删改查，可以通过两套 API 实现：

ros::NodeHandle

ros::param

1.参数服务器新增(修改)参数

2.参数服务器获取参数

3.参数服务器删除参数

>>> 2.3.2 参数操作(C++和Python)

90

1.参数服务器新增(修改)参数

2.参数服务器获取参数

3.参数服务器删除参数

01

02

03

/* 参数服务器操作之新增与修改(二者API一样)_C++实现: 在 roscpp 中提供了两套 API 实现参数操作
ros::NodeHandle setParam("键",值)
ros::param set("键","值") 示例:分别设置整形、浮点、字符串、bool、列表、字典等类型参数 修改(相同的键,不同的值)
*/

/* 参数服务器操作之查询_C++实现:
ros::NodeHandle
param(键,默认值)
getParam(键,存储结果的变量)
getParamCached(键,存储结果的变量)
getParamNames(std::vector<std::string>)
hasParam(键)
searchParam(参数1, 参数2)
ros::param 与 NodeHandle 类似
*/

/* 参数服务器操作之删除_C++实现:
ros::NodeHandle deleteParam("键") 根据键删除参数, 删除成功, 返回 true, 否则(参数不存在), 返回 false
ros::param del("键") 根据键删除参数, 删除成功, 返回 true, 否则(参数不存在), 返回 false
*/

04

常用命令

>>> 2.4 常用命令

92

机器人系统中启动的节点少则几个，多则十几个、几十个，不同的节点名称各异，通信时使用话题、服务、消息、参数等等都各不相同，一个显而易见的问题是：当需要自定义节点和其他某个已经存在的节点通信时，如何获取对方的话题、以及消息载体的格式呢？

roscpp

rostopic

rosservice

rosmmsg

rossrv

rosparam

操作节点

操作话题

操作服务

操作msg消息

操作srv消息

操作参数

文件操作命令是静态的，操作的是磁盘上的文件，而上述命令是动态的，在ROS程序启动后，可以动态的获取运行中的节点或参数的相关信息。

>>> 2.4.1 rosnode

93

rosgen 是用于获取节点信息的命令

- rosgen ping 测试到节点的连接状态
- rosgen list 列出活动节点
- rosgen info 打印节点信息
- rosgen machine 列出指定设备上节点
- rosgen kill 杀死某个节点
- rosgen cleanup 清除不可连接的节点

>>> 2.4.2 rostopic

94

rostopic包含rostopic命令行工具，用于显示有关ROS主题的调试信息，包括发布者，订阅者，发布频率和ROS消息。它还包含一个实验性Python库，用于动态获取有关主题的信息并与之交互。

- | | |
|------------------------------------|------------------------------|
| ➤ rostopic bw 显示主题使用的带宽 | ➤ rostopic info 显示主题相关信息 |
| ➤ rostopic delay 显示带有 header 的主题延迟 | ➤ rostopic list 显示所有活动状态下的主题 |
| ➤ rostopic echo 打印消息到屏幕 | ➤ rostopic pub 将数据发布到主题 |
| ➤ rostopic find 根据类型查找主题 | ➤ rostopic type 打印主题类型 |
| ➤ rostopic hz 显示主题的发布频率 | |

>>> 2.4.3 rosmmsg

95

rosmmsg是用于显示有关 ROS消息类型的 信息的命令行工具。rostopic bw 显示主题使用的带宽。

- rosmmsg show 显示消息描述
- rosmmsg info 显示消息信息
- rosmmsg list 列出所有消息
- rosmmsg md5 显示 md5 加密后的消息
- rosmmsg package 显示某个功能包下的所有消息
- rosmmsg packages 列出包含消息的功能包

>>> 2.4.4 rosservice

96

rosservice包含用于列出和查询ROSServices的rosservice命令行工具。调用部分服务时，如果对相关工作空间没有配置 path，需要进入工作空间调用 `source ./devel/setup.bash`。

- rosservice args 打印服务参数
- rosservice call 使用提供的参数调用服务
- rosservice find 按照服务类型查找服务
- rosservice info 打印有关服务的信息
- rosservice list 列出所有活动的服务
- rosservice type 打印服务类型
- rosservice uri 打印服务的 ROSRPC uri

2.4.5 rossrv

97

rossrv是用于显示有关ROS服务类型的信息的命令行工具，与 rosmmsg 使用语法高度雷同。

- `rossrv show` 显示服务消息详情
- `rossrv info` 显示服务消息相关信息
- `rossrv list` 列出所有服务信息
- `rossrv md5` 显示 md5 加密后的服务消息
- `rossrv package` 显示某个包下所有服务消息
- `rossrv packages` 显示包含服务消息的所有包

2.4.5 rosparam

98

rosparam包含rosparam命令行工具，用于使用YAML编码文件在参数服务器上获取和设置ROS参数。

- `rosparam set` 设置参数
- `rosparam get` 获取参数
- `rosparam load` 从外部文件加载参数
- `rosparam dump` 将参数写出到外部文件
- `rosparam delete` 删除参数
- `rosparam list` 列出所有参数

05

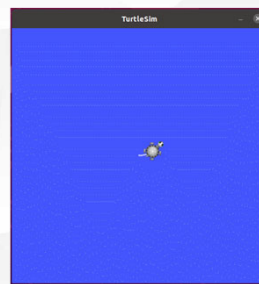
通信机制实操



2.5.1 实操01_话题发布

100

需求描述:编码实现乌龟运动控制，让小乌龟做圆周运动。



实现分析:

乌龟运动控制实现，关键节点有两个，一个是乌龟运动显示节点 `turtlesim_node`，另一个是控制节点，二者是订阅发布模式实现通信的，乌龟运动显示节点直接调用即可，运动控制节点之前使用的是 `turtle_teleop_key` 通过键盘控制，现在需要自定义控制节点。

控制节点自实现时，首先需要了解控制节点与显示节点通信使用的话题与消息，可以使用 `ros` 命令结合计算图来获取。

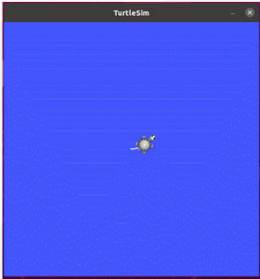
了解了话题与消息之后，通过 C++ 或 Python 编写运动控制节点，通过指定的话题，按照一定的逻辑发布消息即可。

>>> 2.5.1 实操01_话题发布

101

需求描述:编码实现乌龟运动控制,让小乌龟做圆周运动。

TurtleSim



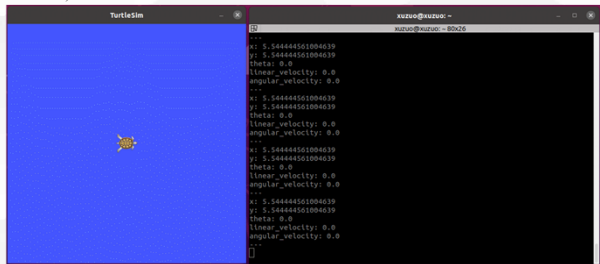
实现流程:
① 通过计算图结合ros命令获取话题与消息信息。
② 编码实现运动控制节点。
③ 启动 roscore、turtlesim_node 以及自定义的控制节点,查看运行结果。

>>> 2.5.2 实操02_话题订阅

102

需求描述:已知turtlesim中的乌龟显示节点,会发布当前乌龟的位姿(窗体中乌龟的坐标以及朝向),要求控制乌龟运动,并时时打印当前乌龟的位姿。

TurtleSim



>>> 2.5.2 实操02_话题订阅

103

需求描述:已知turtlesim中的乌龟显示节点,会发布当前乌龟的位姿(窗体中乌龟的坐标以及朝向),要求控制乌龟运动,并时时打印当前乌龟的位姿。

实现分析:
1.需要启动乌龟显示以及运动控制节点并控制乌龟运动。
2.通过ROS命令,来获取乌龟位姿发布的话题以及消息。
3.编写订阅节点,订阅并打印乌龟的位姿。

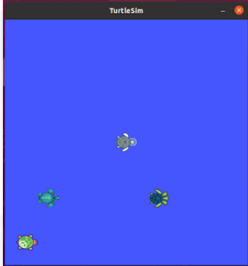
实现流程:
1.通过ros命令获取话题与消息信息。
2.编码实现位姿获取节点。
3.启动 roscore、turtlesim_node、控制节点以及位姿订阅节点,控制乌龟运动并输出乌龟的位姿。

>>> 2.5.3 实操03_服务调用

104

需求描述:编码实现向 turtlesim 发送请求,在乌龟显示节点的窗体指定位置生成一乌龟,这是一个服务请求操作。

TurtleSim



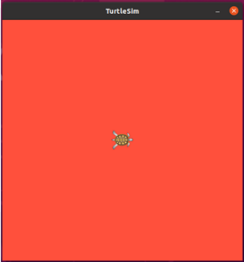
实现分析:
1.需要启动乌龟显示节点。
2.通过ROS命令,来获取乌龟生成服务的名称以及服务消息类型。
3.编写服务请求节点,生成新的乌龟。

实现流程:
1.通过ros命令获取服务与服务消息信息。
2.编码实现服务请求节点。
3.启动 roscore、turtlesim_node、乌龟生成节点,生成新的乌龟。

>>> 2.5.4 实操04 参数设置

105

需求描述: 修改turtleSim乌龟显示节点窗体的背景色, 已知背景色是通过参数服务器的方式以 rgb 方式设置的。



实现分析:

1.需要启动乌龟显示节点。

2.通过ROS命令, 来获取参数服务器中设置背景色的参数。

3.编写参数设置节点, 修改参数服务器中的参数值。

实现流程:

1.通过ros命令获取参数。

2.编码实现参数设置节点。

3.启动 roscore、turtleSim_node 与参数设置节点, 查看运行结果。

06



通信机制比较

>>> 2.6 通信机制比较

107

三种通信机制中, 参数服务器是一种数据共享机制, 可以在不同的节点之间共享数据, 话题通信与服务通信是在不同的节点之间传递数据的, 三者是ROS中最基础也是应用最为广泛的通信机制。

话题通信和服务通信有一定的相似性也有本质上的差异, 二者的实现流程是比较相似的, 都是涉及到四个要素:

- 要素1: 消息的发布方/客户端(Publisher/Client)
- 要素2: 消息的订阅方/服务端(Subscriber/Server)
- 要素3: 话题名称(Topic/Service)
- 要素4: 数据载体(msg/srv)

可以概括为: 两个节点通过话题关联到一起, 并使用某种类型的数据载体实现数据传输。

>>> 2.6 通信机制比较

108

话题通信和服务通信的差异

	Topic(话题)	Service(服务)
通信模式	发布/订阅	请求/响应
同步性	异步	同步
底层协议	ROSTCP/ROSUDP	ROSTCP/ROSUDP
缓冲区	有	无
实时性	弱	强
节点关系	多对多	一对多(一个 Server)
通信数据	msg	srv
使用场景	连续高频的数据发布与接收: 雷达、里程计	偶尔调用或执行某一项特定功能: 拍照、语音识别

2.7 本章小结



本章主要介绍了ROS中最基本的也是最核心的通信机制实现：话题通信、服务通信、参数服务器。掌握本章内容后，基本上就可以从容应对ROS中大部分应用场景了。

THANKS

